

Industrial QC Vision System

Industrial-Grade Technical Documentation
System Architecture, Inspection Pipelines & Optimization

PySide2 · ONNXRuntime + OpenVINO · OpenCV · Plotly · SciPy · Qt5 Desktop

Version: 2.0 | Last Updated: May 2026

1. System Overview

The Industrial QC Vision System is an offline Qt5 desktop application (PySide2). Two inspection modes — surface defect detection and dimensional measurement — share a single window. No web server, no internet dependency. Results are rendered as HTML inside Qt's QWebEngineView. All heavy processing runs in a QThread worker.

This document covers all six modules. Sections 3 and 5 give full detail on the two updated pipelines, including side-by-side algorithm comparisons against the previous version.

Module	Framework / Key Libs	Primary Role
GUI.py	PySide2 (Qt5) + QWebEngineView	Desktop window: 3 tabs, QThread worker, ROI editor, HTML results — UNCHANGED
defect_pipeline.py	ONNXRuntime + OpenCV + NumPy	YOLO patch inference, vectorised postprocess, Canny ROI analysis — UPDATED
advanced_analysis.py	Plotly + SciPy + NumPy	8 analytics views rendered as Plotly HTML — UNCHANGED
measurement_pipeline.py	OpenCV + NumPy + SciPy (convolve2d)	Vectorised edge scan, Huber line fit, inter-edge distance — UPDATED
config.py	stdlib os	Path registry and UI constants — UNCHANGED
utils.py	OpenCV + Pillow + base64	Image encoding and HTML badge/card helpers — UNCHANGED

2. System Architecture

Industrial QC Vision System — Architecture Overview

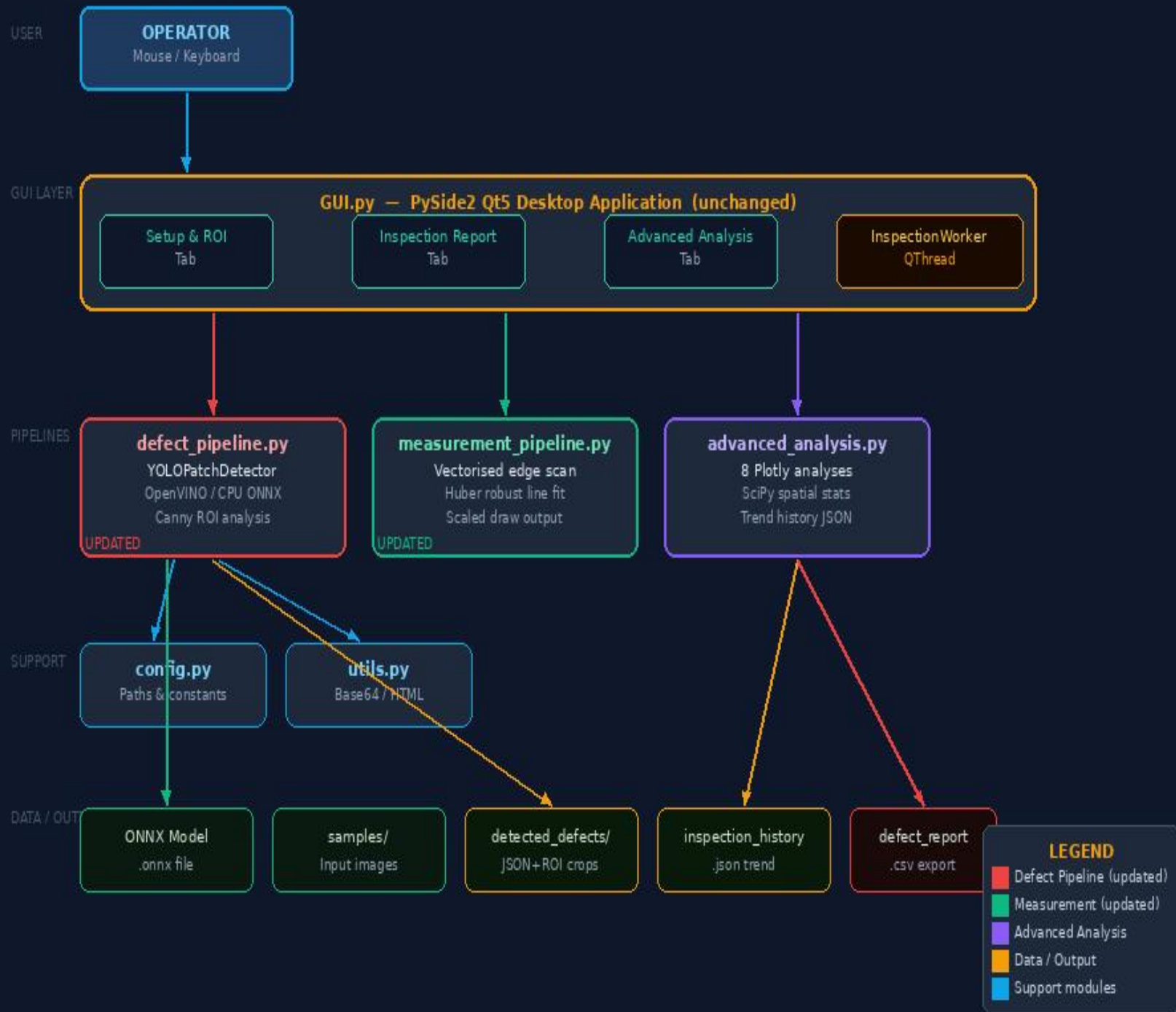


Figure 1 — Full system architecture: layers, data flow, and module relationships

Four layers:

- **User Layer:** Operator interacts with PySide2 widgets — folder pickers, sliders, the ROI editor canvas, action buttons
- **GUI Layer:** GUI.py orchestrates all interaction. Heavy work delegated to InspectionWorker (QThread). Results returned as HTML strings rendered in QWebEngineView
- **Pipeline Layer:** Three independent stateless modules: defect detection, measurement, advanced analytics
- **Data Layer:** Input: samples/ and measurement_samples/. Output: detected_defects/, inspection_history.json, temp CSV exports

3. defect_pipeline.py — v2 Full Reference

CHANGED Two major changes: (1) OpenVINO execution provider added with CPU fallback. (2) postprocess() fully vectorised — class count is now dynamic (output[:,5:]) instead of hardcoded 4 classes.

3.1 What Changed vs v1

Component	V1 (old)	V2 (this version)
Inference provider	CPUExecutionProvider only	OpenVINO provider first, CPU fallback
Class slice in postprocess()	pred[5:9] — hardcoded 4 classes	output[:,5:] — dynamic, reads actual model output width
Confidence computation	Python for-loop per prediction row	Fully vectorised NumPy sigmoid + argmax on full array
ROI contour method	Adaptive threshold, all contours returned	Canny + morph dilate/close/open, largest contour only
Contour min area filter	50 px ²	1 px ² (Canny already suppresses noise)
Box decode rounding	int(x) — truncation	np.round() before int cast

defect_pipeline.py — Inference & Postprocess Data Flow (v2)



Figure 2 — Defect pipeline data flow with v2 changes highlighted

3.2 Inference Provider — OpenVINO with CPU Fallback

The ONNX session now tries providers in priority order:

```
providers = ['OpenVINOExecutionProvider', 'CPUExecutionProvider']
self.session = ort.InferenceSession(model_path, providers=providers)
```

If onnxruntime-openvino is installed and Intel hardware is available, OpenVINO runs the model. Otherwise it silently falls back to CPU. No code change is needed to switch — only the installed package matters.

NOTE pip install onnxruntime-openvino (separate package, in addition to onnxruntime)

3.3 postprocess() — Vectorised, Dynamic Classes

Previous version used a Python for-loop over every prediction row and hardcoded the class slice as pred[5:9]. The v2 version processes the entire output array in one NumPy pass:

Step	NumPy Operation	Detail
Objectness	<code>sigmoid(output[:,4])</code>	Vectorised $1/(1+\exp(-x))$ on entire column at once
Class scores	<code>sigmoid(output[:,5:])</code>	Dynamic — works for any number of classes
Best class	<code>np.argmax(cls_scores, axis=1)</code>	Per-row winner index in single call
Final conf	<code>obj_conf × class_confs[arange, class_ids]</code>	Element-wise multiply, no loop
Threshold mask	<code>final_conf >= confidence_threshold</code>	Boolean mask, one NumPy op
Box decode	<code>cx±w/2, cy±h/2</code> on masked subset	Vectorised on surviving rows only
Coord offset	<code>x_start+round(x), y_start+round(y)</code>	Converts patch-local to full-image coords

The critical fix: `output[:,5:]` replaces `output[row,5:9]`. The model output width determines the class count automatically. Retraining with a different number of classes requires no code change.

ISSUE `stride_ratio=0.01` in `GUI.py get_detector()` still produces only 13px patch overlap. Not fixed in this version. Set to 0.25–0.5 for reliable boundary coverage.

3.4 `process_roi_contours()` — Canny + Morphological

The ROI analysis algorithm was completely replaced. Adaptive-threshold approach is gone.

Stage	V1 — Adaptive Threshold	V2 — Canny + Morphological
Blur	GaussianBlur 5×5	GaussianBlur 5×5
Binarise	<code>adaptiveThreshold (GAUSSIAN_C, blk=11, C=2)</code>	Canny: <code>low=0.6×median, high=1.3×median</code>
Connect edges	(none)	<code>Dilate(ellipse 5×5, iter=1) → MORPH_CLOSE(iter=2)</code>
Cleanup	(none)	<code>MORPH_OPEN(ellipse 3×3, iter=1)</code>
Find contours	<code>RETR_EXTERNAL, filter area > 50 px²</code>	<code>RETR_EXTERNAL, filter area > 1 px²</code>
Selection	Keep ALL valid contours	Keep ONLY the single largest
Draw	All contours, green, text overlay	Largest only, no text overlay
Return contours list	N entries (all valid)	0 or 1 entry

process_roi_contours() — v1 vs v2 Algorithm

V1 — Adaptive Threshold

GaussianBlur 5x5
noise reduction

adaptiveThreshold
GAUSSIAN_C, blk=11, C=2

findContours
RETR_EXTERNAL

Filter: area > 50px²
keep ALL valid contours

Draw ALL contours
green, thickness 2

Return: N contours
full list in output

VS

V2 — Canny + Morphological

GaussianBlur 5x5
noise reduction

Canny
thresholds: 0.6v, 1.3v (median)

Dilate + MORPH_CLOSE
ellipse 5x5, iter 1+2

MORPH_OPEN cleanup
ellipse 3x3, iter 1

findContours + LARGEST
filter > 1px², keep biggest

Draw LARGEST only
Return: 0 or 1 contour

Figure 3 — ROI contour algorithm: adaptive threshold (v1) vs Canny + morphological (v2)

Canny Threshold Selection

Thresholds computed from local image statistics rather than being fixed:

```
v = np.median(blur)
edges = cv2.Canny(blur, int(0.6 * v), int(1.3 * v))
```

This adapts to actual ROI brightness, making edge detection more robust across different lighting and material surfaces.

Morphological Connection

After Canny, dilate+close bridges gaps in the edge map before contour extraction — important for defects with discontinuous edges (scratches, cracks). The open step removes small noise specks introduced by dilation.

Largest-Contour Selection

Only the largest contour by area is kept. The return dict preserves the same keys (contour_count, total_contour_area, contours, contour_visualization) so GUI.py and advanced_analysis.py require no changes. contour_count will be 0 or 1.

3.5 Unchanged Methods

- **preprocess_patch():** resize 1280×1280, float32/255, CHW, batch dim — identical
- **compute_patch_grid():** ceiling-division grid, edge clamping — identical
- **apply_nms():** cv2.dnn.NMSBoxes, OpenCV version safety — identical
- **expand_bbox():** symmetric expansion clamped to image bounds — identical
- **draw_bbox_on_image():** seeded random colors, rectangle + label — identical
- **save_defects():** ROIs, contour images, metadata JSON, annotated image — identical
- **detect():** grid → patch → infer → postprocess → NMS — identical

4. GUI.py — Desktop Application (unchanged)

GUI.py was not modified. Summary of key structure:

4.1 Three-Tab Structure

Tab	Contents
Setup & ROI	Folder pickers, confidence slider (0.10–0.90), ROIEditorWidget with interactive point placement, MeasurementStandardsDialog
Inspection Report	Run buttons (Measurement / Defect / Full), QWebEngineView for HTML results, CSV export
Advanced Analysis	8 analysis buttons, severity threshold slider, QWebEngineView for Plotly charts

4.2 get_detector() — Parameters Passed to defect_pipeline.py

```
YOLOPatchDetector(  
    model_path      = MODEL_PATH,  
    patch_size      = 1280,  
    stride_ratio    = 0.01,      # 13px overlap – see Issue #1  
    confidence_threshold = 0.4,  
    nms_threshold   = 0.45,  
    class_names     =  
    ['scratch', 'dent', 'crack', 'stain', 'missing_part', 'deformation'],  
    expansion_ratio  = 2.0        # 2x in GUI vs 4x pipeline default  
)
```

4.3 InspectionWorker (QThread)

Mode	Signal	Calls
measurement	finished(html, csv)	run_measurement_with_comparison()
defect	finished_with_defects(html, csv, results)	run_defect_only()
full	finished_with_defects(html, csv, results)	run_full_inspection_with_comparison()

4.4 ROI Editor Classes

Class	Inherits	Role
MovablePoint	QGraphicsEllipseItem	Numbered draggable point with embedded text label
ROIScene	QGraphicsScene	Click-add/drag-move/click-delete modes; dashed connector lines between points
ROIEditorWidget	QWidget	Wraps scene+view in QSplitter; toolbar with QPushButton mode selector; Y-alignment tools
MeasurementStandardsDialog	QDialog	QTableWidget for per-pair standard+tolerance; global fallback spinbox

ISSUE `_on_threshold_changed()` passes `self._preload_defect_silently()` with `()` inside `QTimer.singleShot` — executes immediately instead of as a scheduled callback. Not fixed in this version.

5. measurement_pipeline.py — v2 Full Reference

CHANGED Two components replaced: (1) `process_roi()` → `process_roi_vectorised()` using NumPy batch ops and `scipy convolve2d`. (2) `fit_line_and_remove_outliers()` → `fit_line_robust()` using `cv2.fitLine(DIST_HUBER)`. `SAMPLING_STEP` changed 10→15.

5.1 What Changed vs v1

Function	V1 (old)	V2 (new)
Edge extraction	<code>process_roi()</code> — Python for-loop, <code>cv2.resize</code> per strip	<code>process_roi_vectorised()</code> — NumPy batch all strips, <code>scipy convolve2d</code>
Gradient kernel	Computed per-strip inside loop	<code>[0.5,0.5,-0.5,-0.5]</code> conv kernel on full strip matrix
Line fitting	<code>fit_line_and_remove_outliers()</code> — RANSAC 100 trials	<code>fit_line_robust()</code> — <code>cv2.fitLine(DIST_HUBER)</code> + single NumPy distance pass
SAMPLING_STEP	10 px	15 px (fewer strips, faster)
Backward compat	N/A	<code>fit_line_and_remove_outliers()</code> kept but not called by <code>run_pipeline()</code>
Standalone test	No <code>__main__</code> block	<code>__main__</code> block: <code>test.bmp</code> + centers + timing printout

measurement_pipeline.py — v1 vs v2 Algorithm Comparison



Figure 4 — Measurement pipeline: old vs new approach, shared components, and performance gains

5.2 Configuration Constants

Constant	Value	Changed?	Meaning
SAMPLING_STEP	15 px	YES (was 10)	Vertical stride between scan strips — fewer strips, faster
SCAN_PIXEL_NUM	5 px	no	Height of each scan strip fed to gradient computation
EDGE_THRESHOLD	20	no	Minimum gradient magnitude to accept an edge point
HALF_EDGE_WIDTH	2 px	no	Half-window for left/right mean gradient computation
ROI_WIDTH	250 px	no	Width of processing window centred on each user point
ROI_HEIGHT	200 px	no	Height of processing window centred on each user point

5.3 process_roi_vectorised() — Batch Edge Extraction

Replaces the original process_roi() loop. Produces identical output — two lists of (x,y) float tuples — but eliminates the Python for-loop and per-strip cv2.resize.

Step	Detail
Input slice	gray[middle_rows, x:x+w] — all strips at once, shape (num_strips, w)
Kernel	[0.5, 0.5, -0.5, -0.5] — left-half mean minus right-half mean
Convolution	convolve2d(profiles, kernel, mode=valid) → shape (num_strips, w-3)
Light→dark edge	argmax of positive gradient row → peak column = left edge x
Dark→light edge	argmax of negated gradient row → peak column = right edge x
Threshold	Peak gradient value must be $\geq$ EDGE_THRESHOLD (20) to accept
Output	Two lists of (x,y) float tuples — same interface as v1 process_roi()

scipy convolve2d with mode='valid' processes all strips simultaneously. The kernel [0.5, 0.5, -0.5, -0.5] computes the same gradient as the original HALF_EDGE_WIDTH window averaging — mathematically identical, vectorised execution.

NOTE fit_line_and_remove_outliers() is kept for backward compatibility but is no longer called by run_pipeline(). Superseded by fit_line_robust().

5.4 fit_line_robust() — Huber Robust Fitting

Replaces the 100-trial RANSAC loop. Uses OpenCV's built-in robust fitter with Huber loss, then refits with least-squares on the inlier set.

Stage	V1 RANSAC	V2 Huber
Initial fit	Random 2-point sample, up to 100 trials	cv2.fitLine(DIST_HUBER) — single robust fit
Inlier selection	Count inliers per trial, keep best trial	Single pass: perp dist < threshold (3px)
Refit	Least-squares on best_inliers	Least-squares on inliers (same logic)
Vertical edge	std(x)<2 → vx=0,vy=1	std(x)<2 → vx=0,vy=1 (identical)
Fallback	Return all valid if < 2 inliers	Return all valid if < 2 inliers (identical)
Output format	(inlier_list, (vx,vy,x0,y0))	(inlier_list, (vx,vy,x0,y0)) — identical interface

The output interface (inlier_list, (vx, vy, x0, y0)) is identical to the original. All downstream code in `measure_and_draw_distances_scaled()` works without modification.

5.5 `run_pipeline()` — Entry Point

Unchanged in logic. Now calls the new functions:

```
for cx, cy in centers:
    roi = (cx-125, cy-100, 250, 200)
    e1, e2 = process_roi_vectorised(gray, roi)      # NEW
    e1_inliers, e1_line = fit_line_robust(e1, 3.0)  # NEW
    e2_inliers, e2_line = fit_line_robust(e2, 3.0)  # NEW
```

All processing at original image resolution. Scaling applied only at draw time. Returns (final_image, distances) where distances are in original pixel units.

5.6 Unchanged Functions

- **`measure_and_draw_distances_scaled()`**: Scales coords, draws green/blue edge lines, alternating cyan/lime distance annotations, diamonds, arrows, semi-transparent text background
- **`draw_vertical_line_scaled()`**: Draws fitted edge line using mean-x of inlier points
- **`resize_and_adjust_points()`**: `cv2.resize` + proportional coordinate scaling
- **`get_line_x_at_y()`**: Solves line parametric equation for x at a given y
- **`draw_horizontal_distances_with_centers()`**: Legacy drawing function — kept for compatibility

5.7 New: Standalone Test Block

Added `__main__` block (not present in v1) for direct testing:

```
if __name__ == '__main__':
    img = cv2.imread('test.bmp')
    centers = [(2100,3000), (3200,3000), ...]
    result, distances = run_pipeline(img, centers, display_scale_percent=7)
    print(f'Processing time: {end-start:.3f}s')
    cv2.imshow('Industrial Measurement', result)
```

6. advanced_analysis.py — Reference (unchanged)

Button / Function	Charts	Key Inputs
Pareto Analysis	2× Bar + cumulative line	Class frequency, total area per class
Morphological	Stats table + 2× Box plots	Circularity, elongation — first contour only
Spatial Pattern	Gaussian heatmap + NN ratio	Normalised center_x/y, image shape
Severity	KPI cards + Bar + Box + Pie	Type weights × (log area + confidence/2)
Trend	Dual line chart (defects + area)	inspection_history.json — appended each call
Defect Density	Bar + Pie + KPI cards	pixel_to_cm2=0.001 conversion factor
Root Cause	Zone table + Bar + Pie	Left/Right/Top/Bottom quadrant split
Box Plots	2× Box plots	Area distribution + circularity by class

ISSUE `extract_defect_data()` uses `contour_details[0]` only for circularity/perimeter. In v2 this is now always the only contour — behaviour unchanged but previously could silently ignore contours 1–N.

ISSUE `inspection_history.json` written to `os.getcwd()`, not `BASE_DIR`. File location depends on launch directory.

7. config.py and utils.py — Reference (unchanged)

7.1 config.py

- **BASE_DIR:** `os.path.dirname(os.path.abspath(__file__))` — all paths relative to this
- **SAMPLES_DIR / MEASUREMENT_DIR:** Default input folders
- **MODEL_PATH:** `BASE_DIR/models/best_4_1280.onnx`
- **DISPLAY_SCALE_PERCENT:** 7 — display scale for measurement output image
- **SERVER_PORT / SERVER_NAME:** Unused Gradio-era leftovers — safe to remove

7.2 utils.py

- **numpy_to_base64(img, size):** BGR→RGB, PIL thumbnail LANCZOS, BytesIO JPEG, returns data:image/jpeg;base64,... URI
- **create_defect_badge(type, conf):** HTML span with class color: scratch=#EF4444, dent=#F59E0B, crack=#DC2626, stain=#8B5CF6, missing_part=#EC4899, deformation=#06B6D4
- **create_kpi_card(label, value, unit, color):** Dark gradient div with left stripe. Colors: blue/red/green/yellow

8. Known Issues — v2 Status

Issue	File	v2 Status	Fix
stride_ratio=0.01 → only 13px patch overlap	GUI.py	NOT FIXED	Set stride_ratio=0.25–0.5 in get_detector()
pred[5:9] hardcoded 4 classes	defect_pipeline.py	FIXED — output[:,5:] dynamic	No action needed
CPU-only ONNX inference	defect_pipeline.py	FIXED — OpenVINO provider added	Install onnxruntime-openvino
_on_threshold_changed() calls fn with ()	GUI.py	NOT FIXED	Remove () → pass as callback reference
trend_analysis writes to os.getcwd()	advanced_analysis.py	UNCHANGED	Use BASE_DIR from config.py
QApplication.processEvents() blocks UI	GUI.py	UNCHANGED	Move _ensure_defect_results to QThread
No startup model existence check	GUI.py	UNCHANGED	Check MODEL_PATH in MainWindow.__init__
SERVER_PORT/SERVER_NAME unused leftovers	config.py	UNCHANGED	Remove Gradio-era constants

FIXED issues are resolved in the updated defect_pipeline.py. NOT FIXED and UNCHANGED issues remain in GUI.py or advanced_analysis.py.

9. Directory Structure

```
project_root/
├── GUI.py
├── defect_pipeline.py      ← updated v2
├── advanced_analysis.py
├── measurement_pipeline.py ← updated v2
├── config.py
├── utils.py
├── models/
│   └── best_4_1280.onnx    ← required
├── samples/                ← defect input images
├── measurement_samples/    ← measurement images
├── detected_defects/       ← auto-created
```

```

|   |   | rois/
|   |   | contours/
|   |   | annotated_image.jpg
|   |   | defects_metadata.json
|   | inspection_history.json ← created by trend_analysis()

```

10. Dependencies

Package	Used In	Notes
PySide2	GUI.py	Qt5 bindings incl. QWebEngineWidgets for HTML rendering
onnxruntime	defect_pipeline.py	CPU build; swap for onnxruntime-gpu for CUDA
onnxruntime-openvino	defect_pipeline.py	Optional — enables OpenVINO provider (NEW in v2)
opencv-python	All pipelines	Core image processing + NMS drawing
numpy	All modules	Array ops throughout
Pillow	utils.py	PIL Image for thumbnail + JPEG encode
plotly	advanced_analysis.py	All charts rendered as self-contained HTML
scipy	advanced_analysis.py + measurement_pipeline.py	gaussian_filter, distance_matrix, convolve2d

```

pip install PySide2 onnxruntime opencv-python numpy Pillow plotly scipy
pip install onnxruntime-openvino # optional - enables OpenVINO provider

```

11. Launch

```
python GUI.py
```

Window opens at 1350×900. Background detector preload starts 500ms after launch. Model must exist at models/best_4_1280.onnx before the first defect run.